RSA alg. (1978) // all msg's, all keys are numbers.

1. Pick two very large primes, p, q.
   compute $n = p \cdot q$.
2. Pick a small odd number e s.t.
   $\gcd(e, (p-1)(q-1)) = 1$. // e and // $(p-1)(q-1)$ are // co-prime
3. compute d s.t.
   $ed \bmod (p-1)(q-1) = 1$.
4. public key: (e, n) //
   private key: (d, n). // // destroy p, q.

Encryption: $C = (M^e \bmod n)$.

Decryption: $M = (C^d \bmod n)$

Worries: ① how many prime #'s?

density.

Euler's prime density $\sim \frac{n}{\ln n}$.

$\frac{n}{\ln n}$ = # of primes with the range $1 \sim n$.

② security of RSA relies on:
there is no efficient (poly-time) alg

to solve the following problem:

FACT { Given: n
       Want: two primes p, q s.t. $n = p \cdot q$.

↳ large # factorization.

obs 1:
~~Q1~~ of ②: the security of RSA relies on an assumption (not a fact).

It's an open problem whether RSA is secure.

// FACT $\in$ NP but we don't know whether it's NP-complete. (next wk).

// Recall: time complexity is measured (is a function) on
// input size (NOT input value.)

Obs: I have the following brute-force
alg to solve FACT:

input n

Enumerate all $p, q \leq n$ s.t

check $n == p \cdot q$ and

p, q are primes;

If yes → ret p, q

primetesting can be solved in poly-time. (IIT Kanpur)

Time complexity of my brute-force alg.:

① n^2 iterations on $1 \le p, q \le n$.

② Assume that primetesting can be done in Constant time;

③ total running time: $O(n^2) = O(2^N)$

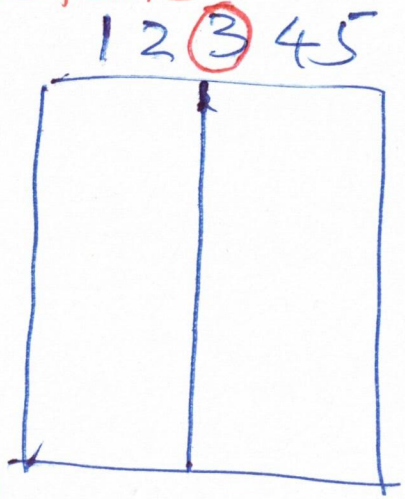
size of input n ↑

$N = \log_2 n$ exp!

Nondet. poly-time algs. (key concept).

Nondet. in CS:

① . Nondet. systems



What is the next right ?

④ → ④ or

③ → ②

nondeterministic



why?

because part of the interface is hidden from you.

② . Nondet. Algs.

Universe